# Table of Contents

# Monthly Progress Report — MSc DFT AI Assistant (Modules #4–#7)

**Project:** AI-Driven Assistant for Applicants to NUS MSc in Digital Financial Technology (FT5007 internal capstone) **Scope owned:** MVP items #4–#7 — Application Checklist, Status Tracking & Reminders, Program Comparison, Course & Career Recommendation
**Reporting period:** June 2026 (from a standing start to a working, tested system) **Status:** On track · 267 automated tests passing · core features demonstrable end-to-end

---

## 1. Executive Summary

Over this period I built, from zero, the four assistant agents I am responsible for (#4–#7) into a **working, tested, offline-capable system**, and then deepened three of them to production-shaped quality. The system now:

- Generates **personalised application checklists** and identifies missing documents (#4);
- Translates **application status**, plans milestone reminders, and **sends real email notifications** over SMTP (#5);
- Produces **objective, sourced program comparisons** that cleanly separate verified facts from AI synthesis and never make unsupported rankings (#6);
- Recommends **courses and career paths** using a progress-aware engine where rules build a candidate set and the LLM selects within it under strict validation (#7).

The work was delivered under a disciplined engineering process (specification → plan → test-driven implementation → independent code review), resulting in **267 passing automated tests, a 12/12 evaluation scorecard, zero hardcoded secrets, and full offline operability**. Real-world grounding was achieved by integrating **live NUSMods course data** and **verified competitor program data**, and by **successfully sending a real reminder email** through a configured mail server.

## 2. Responsibilities & Context

The overall product is a supervisor-plus-specialist-agent system intended to replace a large share of Level-1/Level-2 human support across the student lifecycle. My remit is four of the nine MVP capabilities:

| # | Module | Capability | Primary intents |
|---|---|---|---|
| 4 | Checklist | Personalised document checklist + missing-item detection | `generate_application_checklist`, `check_missing_documents` |
| 5 | Tracker | Status translation + proactive reminders (now with real email) | `get_application_status`, `configure_reminders` |
| 6 | Comparator | Objective program comparison + goal-fit narrative | `compare_programs` |
| 7 | Navigator | Course recommendation + career/skill-gap guidance | `recommend_courses`, `recommend_career_path` |

A guiding architectural principle was adopted and held throughout: **decisions are made by deterministic rules; the LLM only phrases (and, where introduced, selects within guardrails)**. This keeps outputs reproducible, auditable, testable offline, and free of fabrication — properties the project's compliance requirements demand.

---

## 3. What Was Built

### Phase 1 — Foundation: four working agents

- Established the shared contract layer: a single `UserProfile` input model and a uniform `AgentResponse` envelope, so every agent and every consumer speaks one language.
- Implemented deterministic rule engines for all four agents, with the LLM layer degrading gracefully to templates when no API key is configured.
- Mocked external systems (admissions/CRM) behind API-shaped interfaces so they can be swapped for real integrations later.
- Added supporting infrastructure beyond the core four: a **natural-language admin authoring tool** (edit data files in plain language with validation, diff, archival, audit

log, and rollback) and a **graded data-refresh pipeline** that pulls **real NUSMods course data**.

### Phase 2 — Research foundations (directions B → A → C)

- **B — Quality evaluation framework:** a deterministic scorecard (`eval.runner`) over a hand-labelled case set, used as a regression gate. **12/12 baseline maintained throughout the month.**
- **A — Retrieval + threshold calibration:** a curated knowledge base with a **pluggable retriever** (offline BM25 default; embedding backend when configured). Integrated a local **Ollama** `nomic-embed-text` model, empirically compared BM25 vs. embedding (BM25 accuracy 1.0 vs. embedding 0.92), and calibrated **per-backend thresholds** so policy-sensitive questions safely escalate to humans.
- **C — Personalisation taxonomy + fairness:** an ESCO/O*NET-referenced skill taxonomy and a pluggable skill matcher, with a **consent gate** (opt-out → generic guidance) and a **fairness invariant** (the capability text used for inference excludes nationality/country).

### Phase 3 — Feature deepening (the month's main delivery)

**#6 Comparator v3 — fact/synthesis separation.** Upgraded each comparison cell to a three-state model (`verified` / `unknown` / `synthesis`); restructured the engine so each row's **facts** are kept separate from **AI synthesis** (fit scores, narrative); expanded the comparison table to the 8 PDF-mandated dimensions; and replaced a prompt-only safeguard with a **deterministic anti-ranking guard** that rejects any "X is better than Y" language and falls back to a safe template. A final review caught and fixed a real bypass (plural phrasing such as "best programmes").

**#5 Tracker — real notification engine with email.** Turned reminders into a true proactive-notification engine: a `configure_reminders` action (channels / frequency / per-milestone mute), **daily-digest grouping**, and a **deduplicated delivery model** (`due_now` preview + `dispatch_due` that records what was sent so nothing fires twice). Introduced a **pluggable** `Notifier` seam and implemented real **SMTP email** (stdlib, with implicit-SSL support for providers like 163/QQ). Credentials are read from environment/local file only (never committed). **A real reminder email was successfully delivered**

**end-to-end** during verification; with no mail server configured, the system degrades to record-only and stays fully offline.

**#7 Navigator — progress-aware, LLM-constrained recommendation.**

Replaced the fixed role→module table with a **rules-build-candidates / LLM-selects-within-them** design: rules assemble a candidate pool (curated role modules ∪ gap-addressing modules, minus already-completed), the LLM picks an ordered shortlist **only from candidate codes** (invented codes are dropped; offline/invalid → deterministic ranking). Completed courses now (a) shrink skill gaps via a new module→skill map, (b) are excluded from recommendations and marked "done", and (c) feed accurate, non-double-counted graduation progress. Split the two intents into distinct course- vs. career-focused views, and added a soft warning for unrecognised completed-course codes. A final code review confirmed the guardrail cannot be bypassed and the degrade path is safe, and surfaced one **consent fix** (now applied): under opt-out, the LLM is not consulted at all, so personalised gap data never leaves the system.

---

## 4. Key Metrics

| Metric | Value |
|---|---|
| Automated tests passing | **267 passed, 1 skipped** |
| Test files | 28 |
| Evaluation scorecard (regression gate) | **12 / 12** |
| Production source code | ~5,900 lines (excl. tests) |
| Design documents (specs/plans/designs) | 22 |
| External data integrations | Live NUSMods catalog; verified competitor program dataset (NUS/SMU/NTU/HKUST) |
| Real email delivery | Verified (`SENT: True`) over configured SMTP |
| Hardcoded secrets | 0 (env / gitignored local file only) |
| Offline operability | Full (no network/API key required for agents or tests) |

## 5. Engineering Practices

A consistent, auditable workflow was used for every non-trivial change:

```
Brainstorm (clarify intent, trade-offs)
        ↓
Write specification  → committed design doc
        ↓
Write implementation plan  → bite-sized, test-first tasks
        ↓
Subagent-driven implementation  (TDD: red → green → refactor, per task)
        ↓
Independent code review  → fix-then-ship
        ↓
Commit (one logical change per commit)
```

- **Test-driven development** throughout: a failing test precedes every implementation step.
- **Deterministic, offline tests:** the LLM and all network calls are mocked; the suite never touches a real endpoint.
- **Independent final reviews** (including deep reviews focused on safety guardrails) caught two real defects this month — an anti-ranking bypass (#6) and a consent leak (#7) — both fixed before sign-off.
- **Living documentation:** a single authoritative project overview plus a per-change changelog, kept in sync via an editor reminder hook.

---

## 6. Compliance & Safety Highlights

These directly address the capstone's stated requirements:

- **No fabrication.** Recommended/compared items come only from curated, sourced datasets; the #7 selector validates every LLM choice against a real candidate set; the #6 narrative is guarded against unsupported rankings.
- **Fact vs. inference is explicit.** #6 tags every comparison cell (verified / unknown / synthesis) and separates the fact table from AI synthesis; answers carry an official/advisory/recommendation type.
- **Consent and fairness.** Opt-out suppresses personalised inference end-to-end (including, after this month's fix, not sending it to the LLM); nationality is excluded from skill inference.
- **Safe degradation.** Every external dependency (LLM, embeddings, email) fails safe to a deterministic offline path rather than crashing.
- **Auditability & least exposure.** Admin edits are validated, versioned, and logged; secrets live only in gitignored local files or environment variables.

---

## 7. Risks & Known Limitations

| Item | Note |
|---|---|
| External systems are mocked | Admissions/CRM/SIS integrations are API-shaped mocks; real connection is future work. |
| Completed-course data is self-reported | Students type module codes (soft validation added); a real Student Information System feed is future work. |
| Course catalog is a NUSMods subset | A genuinely-taken module outside our subset is flagged “please verify”, not wrong. |
| Module → skill map is a curated seed | Editorial mapping; can be refined by staff over time. |
| LLM selection requires a key for the richest output | Without a key the system uses deterministic ranking (fully functional, reproducible). |

## 8. Next Steps (candidates)

- Extend the evaluation scorecard to cover #5 and #6 (currently #4 + #7), hardening the regression net.
- Deepen retrieval evaluation with a larger labelled query set and recall@k metrics.
- Wire the module → skill map into the admin authoring tool so staff can maintain it.
- Optional: real Student Information System / admissions API integration to replace mocks.

---

*Prepared as a monthly summary of work on MSc DFT assistant modules #4–#7. Supporting detail: see the project overview (`docs/00-project-overview.md`), per-module design docs (`docs/07`–`docs/10`), specs/plans (`docs/superpowers/`), and the changelog (`CHANGELOG.md`).*